



TRAINING PROGRAMME

LLMs & AI Agents

Build, secure, evaluate and deploy applications on large language models

27 HOURS

FOR Developers who want to build applications with large language models



Mohamed Aziz BEN HAHHA

mohamedazizbenhaha.netlify.app

Overview

This programme teaches developers to build applications on large language models and to put them into production responsibly. Participants learn how Transformers and LLMs are trained and where their limits are, write effective prompts and structured outputs, use hosted APIs and open models, build retrieval-augmented generation over their own documents, and give models tools through function calling, agent frameworks and the Model Context Protocol. They then defend the application against prompt injection, evaluate it with test sets and LLM judges, trace its cost and serve it as an API. The programme closes with a final project: an agent application built, evaluated and presented by each team.

At a glance

#	PHASE	FOCUS	HOURS
01	LLM Foundations	Transformers and training, prompting, structured output, APIs and open models	6 h
02	RAG & Agents	Embeddings, vector stores, retrieval and reranking, tool use, agent frameworks	6 h
03	MCP, Security & Evaluation	Model Context Protocol, prompt injection and guardrails, evaluation, tracing, deployment	6 h
04	Final Project	An agent application, evaluated and deployed	6 h
	Buffer	Catch-up, revision, extra lab time	3 h
	Total		27 h

Key facts

AUDIENCE	Developers who want to build applications with large language models
PREREQUISITES	Python programming (course: Python Programming Fundamentals), or equivalent; basic Docker use helps (course: Docker & Kubernetes). No machine learning background needed
DURATION	27 h: 24 h of chapters + 3 h buffer
FORMAT	Short lectures followed by guided labs; one application carried through every phase
LAB ENVIRONMENT	One laptop per participant (16 GB RAM, Python 3, Docker, VS Code, Ollama for open models); API access to a hosted LLM provided by the organiser with a spending cap; no other account or cost
ASSESSMENT	A lab validated at the end of each phase and a final project presented to the group; certificate of completion for participants who complete both
OUTCOME	Participants can build a RAG and tool-using agent application, reduce its exposure to prompt injection, measure its quality and cost, and deploy it as a containerised API

PHASE 01

6 H

LLM Foundations

Participants learn how LLMs are built, from attention and tokens to pretraining, instruction tuning and preference optimisation, and what reasoning models change. They then write prompts that work, get structured output, and call both hosted APIs and open models running locally. Validated lab: the same extraction task solved with a hosted model and a local open model, returning validated JSON.

#	CHAPTER	CONTENT	FORMAT	HOURS
1.1	Transformers & LLMs	Attention, tokens, pretraining, instruction tuning, RLHF and DPO, reasoning models, model landscape	Lecture + Workshop	3 h
1.2	Prompt Engineering	Patterns, few-shot prompting, structured output	Lecture + Lab	1.5 h
1.3	Using Models	LLM APIs, structured outputs, open models (Ollama, Hugging Face)	Lecture + Lab	1.5 h

PHASE 02

6 H

RAG & Agents

Participants ground answers in their own documents with retrieval-augmented generation, from embeddings and chunking to hybrid retrieval, reranking and RAG evaluation, then let models act through tool use and function calling, planning, and multi-agent frameworks such as LangGraph and CrewAI. Validated lab: a question-answering assistant over a document set that cites its sources, extended into an agent that calls two tools.

#	CHAPTER	CONTENT	FORMAT	HOURS
2.1	Retrieval-Augmented Generation	Embeddings, vector stores, chunking, hybrid retrieval, reranking, RAG evaluation	Lecture + Lab	3 h
2.2	Agents	Tool use, function calling, planning, multi-agent frameworks (LangGraph, CrewAI)	Lecture + Lab	3 h

PHASE 03

6 H

MCP, Security & Evaluation

Participants expose tools through the Model Context Protocol, defend the application against prompt injection and the other OWASP risks for LLM applications, then measure quality with evaluation sets and LLM-as-judge, trace calls and cost, and serve the application as an API in a container. Validated lab: an MCP server used by the agent, an injection attempt blocked by a guardrail, and an evaluation report run before deployment.

#	CHAPTER	CONTENT	FORMAT	HOURS
3.1	MCP	Building and connecting MCP servers and tools	Lecture + Lab	1.5 h
3.2	Guardrails & Security	Prompt injection, OWASP Top 10 for LLM Applications (2025), guardrails	Lecture + Lab	1.5 h
3.3	Evaluation & Deployment	Evaluation sets, LLM-as-judge, tracing, cost; serving as an API in a container	Lecture + Lab	3 h

PHASE 04

6 H

Final Project

Teams of two or three build an agent application for a written brief, reusing the phase labs, then present its quality, security and cost as they would to a product owner.

#	CHAPTER	CONTENT	FORMAT	HOURS
4.1	Brief & Build	Read the brief, design the agent, build RAG over the provided documents, tools and an MCP server	Project	3 h
4.2	Evaluate & Secure	Evaluation set and report, guardrails, tracing and cost per request, containerised API	Project	1.5 h
4.3	Demo & Review	Live demo including an attack attempt; questions and feedback	Project	1.5 h

FINAL PROJECT

A Deployed Agent Application

An agent that answers from a document set with cited sources, acts through tools and an MCP server, is tested against prompt injection, is measured on an evaluation set with its cost per request, and runs as a containerised API.

STACK Python, an LLM API and an open model (Ollama), embeddings and a vector store, LangGraph, MCP, a guardrails library (e.g. NeMo Guardrails), Ragas, Langfuse, FastAPI, Docker

DELIVERABLE Repository with code and evaluation set, evaluation report, live demo